CityTech TTP Summer Bootcamp

The Command Line

2026-06-03

Agenda

1. Review
2. Integrating Changes
3. Merge Conflicts
4. Continue 21 Questions
5. Share your repos
6. Why the Command Line
7. Navigating and Files
8. Getting Help
9. Play to Practice

Review

What are the most important things you remember from yesterday?

Integrating Changes

Three ways git brings one branch's work into another:

- **Fast-forward** — `main` hasn't moved, so its pointer just slides forward. No new commit.
- **Merge** — histories diverged; git makes a **merge commit** that joins them.
- **Rebase** — replays your commits on top of the latest `main` for a **linear** history.

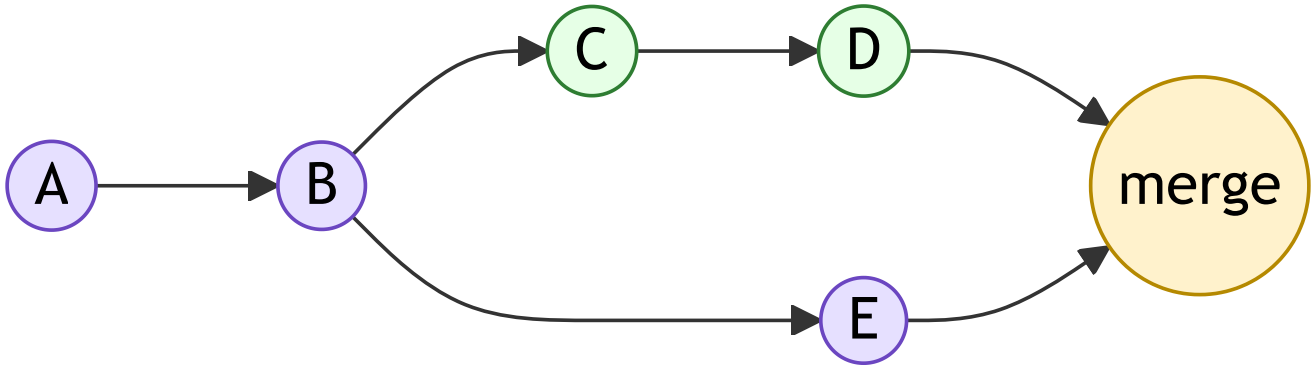
Fast-forward

If `main` has no new commits since you branched, git simply moves the `main` pointer forward to your branch's tip — no merge commit needed.

Green = your branch's commits.

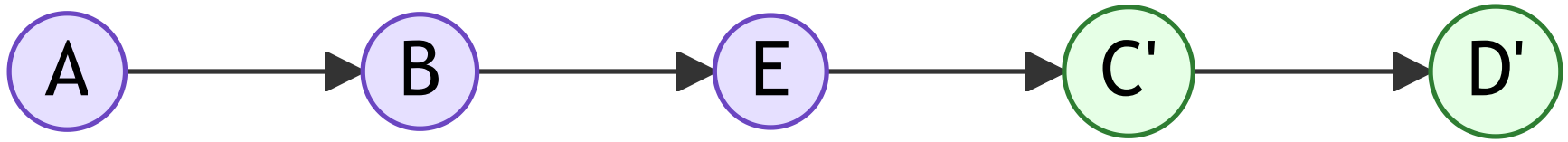
Merge

When **both** branches have new commits, git ties them together with a dedicated **merge commit** (yellow). History keeps both lines visible.



Rebase

Rebase **replays** your branch's commits on top of the latest `main`, rewriting them (`C → C'`) so history stays **linear** — as if you'd started from the newest code.



The 3 Strategies, ELI5

Everyone's writing in one shared notebook:

- **Fast-forward** — no one added pages while you were gone, so yours go right on the end.
- **Merge** — two people wrote at once, so we tape both sets of pages in and add a sticky note: "joined here."
- **Rebase** — you neatly recopy your pages to sit *after* everyone else's newest pages — one clean stack, no sticky note.

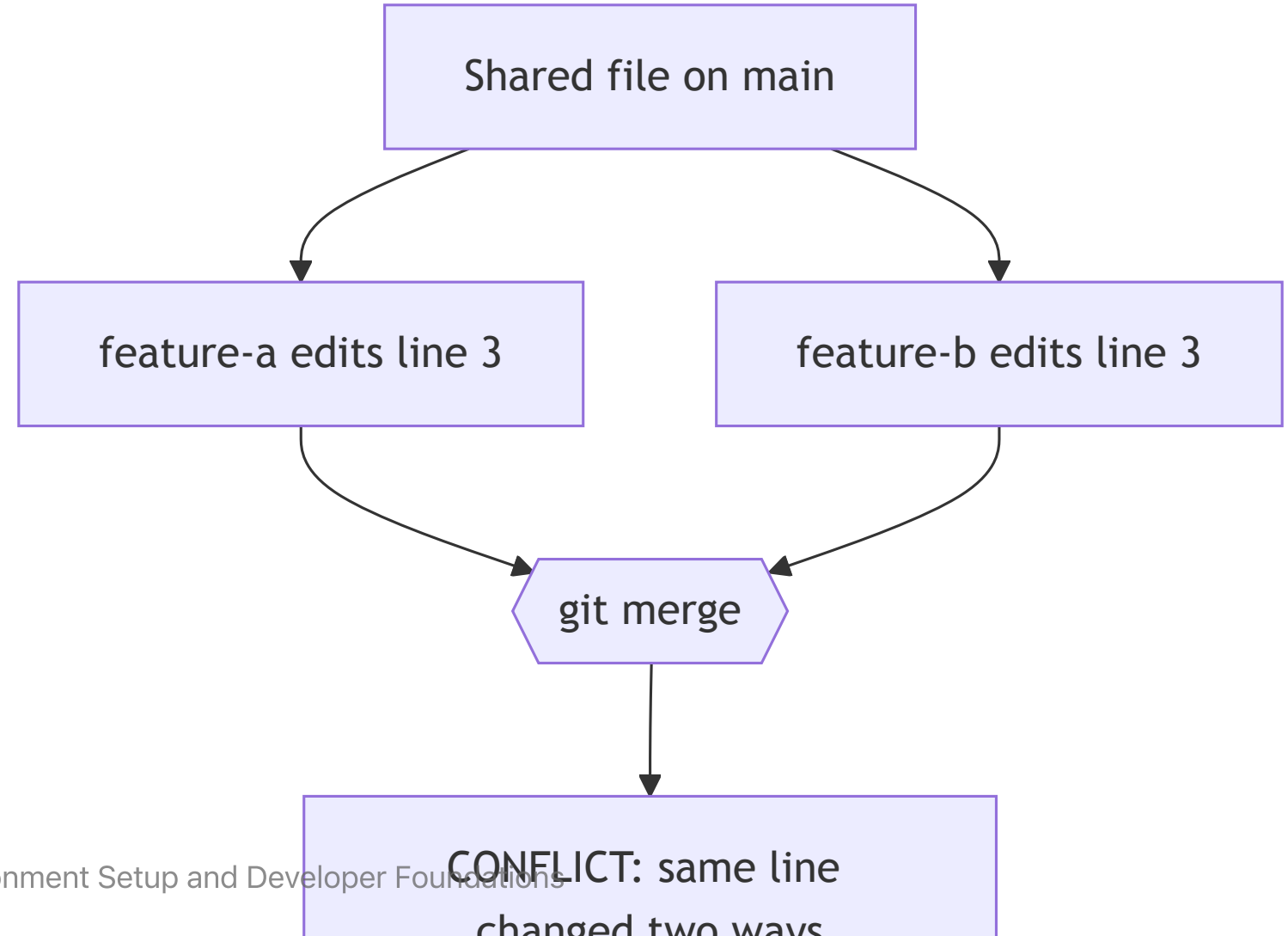
Merge Conflicts

A **merge conflict** happens when two branches change the **same lines** of a file in different ways, and git can't decide which version to keep.

- Common when you and a teammate edit the same file
- git pauses the merge and asks **you** to choose
- They're normal — not a sign you did something wrong

> **No conflict if** you edited different files — or if your changes touch different parts of a file that git can merge automatically.

Why a Conflict Happens



What a Conflict Looks Like

git marks the clashing section right in the file:

```
<<<<<<< HEAD
greeting = "Hi"
=======
greeting = "Hey"
>>>>>>> feature-b
```

- `<<<<<<< HEAD` — your current branch's version
- `=======` — the divider
- `>>>>>>> feature-b` — the incoming branch's version

Resolving a Conflict — Edit It

The most common way: open the file in **VS Code** (or any editor) and choose what to keep.

- Delete the `<<<<<<<` / `=======` / `>>>>>>>` markers, leaving the version you want
- VS Code adds *Accept Current / Incoming / Both* buttons
- …and a 3-way **Merge Editor** for side-by-side resolution

Resolving a Conflict — Take One Side

When you just want one branch's version of the whole file, pick it from the CLI:

```
git checkout --ours   greeting.js   # keep your branch's version
git checkout --theirs greeting.js   # keep the incoming version
```

Resolving a Conflict — Finish or Back Out

Once the file looks right, mark it resolved:

```
git add greeting.js
git commit
```

Or throw away the merge and start over:

```
git merge --abort
```

> `git status` lists every file still in conflict.

Continue 21 Questions Game

https://github.com/jonathan-chin/citytech-ttpr-2026-summer-github-practice-21-questions

Share your repos!

https://forms.gle/vrK7GPxsEPmZ2YvV8

The Command Line

Talking to your computer with text

Why the Command Line?

- It's fast and scriptable once it's in your fingers
- A **GUI** (graphical user interface) is resource-intensive — it needs more powerful, more expensive hardware
- Not every task needs that — a plain terminal runs fine on cheap, minimal machines
- Remote machines (servers) often have no graphical interface at all
- Windows users: do all of this in your **WSL (Ubuntu)** terminal

Anatomy of a Command

A command is usually: `command [options] [arguments]`

```
ls -l /home
# command: ls    option: -l    argument: /home
```

Where Am I? — `pwd`, `ls`

```
pwd             # print working directory (where you are)
ls              # list files here
ls -a           # include hidden files (dotfiles)
ls -l           # long format: permissions, size, date
ls -la          # both at once
```

Moving Around — `cd`

```
cd projects          # into the "projects" folder
cd ..                # up one level
cd ~                 # your home directory
cd /                 # the root of the filesystem
cd -                 # back to the previous directory
```

Paths: `~` = home, `.` = here, `..` = parent.

Absolute paths start at `/`; **relative** paths start from where you are.

"Directory" vs. "Folder"

- They mean the **same thing**: a container that holds files and other directories.
- **Directory** is the older, command-line term — it's why we have `cd` (**c**hange **d**irectory) and `mkdir` (**m**a**k**e **dir**ectory).
- **Folder** is the GUI term, from the file-explorer "folder" icon metaphor.
- Use them interchangeably — on the command line you'll mostly hear **directory**.

Making Things — `mkdir`, `touch`

```
mkdir week1                    # make a directory
mkdir -p week1/exercises       # make nested directories
touch notes.md                 # create an empty file (or update its timestamp)
```

Copying and Moving — `cp`, `mv`

Both follow the same shape:

```
cp source destination     # copy
mv source destination     # move or rename
```

Examples:

```
cp notes.md notes-backup.md     # copy a file
mv notes.md week1/              # move into a folder
mv old.md new.md                # rename
```

Deleting — `rm` (careful!)

```
rm notes.md           # delete a file
rm -r week1-copy      # delete a directory and everything in it
```

⚠️ There is **no Trash or Recycle Bin** on the command line.

Deletes are permanent — read the line twice before you press Enter.

Looking Inside Files

```
cat notes.md          # print the whole file
less notes.md         # scroll a long file (press q to quit)
head notes.md         # first 10 lines
tail notes.md         # last 10 lines
tail -f server.log    # follow a file as it grows
```

Getting Help and Moving Faster

```
man ls            # manual page (q to quit)
ls --help         # quick help for most commands
history           # commands you've run
clear             # clean up the screen
```

- **Tab** completes file and command names
- **↑ / ↓** scroll through past commands
- **Ctrl + C** cancels the running command

When you're stuck:

- Ask other engineers (peers, seniors)
- Read The Manual (**RTM**)
- Ask AI

Play to Practice

1. https://cmdchallenge.com/
2. https://overthewire.org/wargames/bandit/
3. https://web.mit.edu/mprat/Public/web/Terminus/Web/main.html
4. https://github.com/veltman/clmystery